inDrive Technical Interview Prep – Wed 11 Nov, 12:30–14:00 Cairo

Nov 7, 2026 · @Ahmed Hadi

The interview at a glance

Pipeline architecture is 40–50 of the 90 minutes, more than half the interview. It is also where your 20 years of experience counts most. Python and SQL just need to be correct, clean and explained out loud. They don't have to be clever.

Section	Minutes	What they are really testing	What a strong performance looks like
Introductions	10	Communication, confidence, relevance	A 60-second intro that ends on why inDrive (section 6)
Python task	15–20	Can you turn a problem into working code and test it?	Clarify inputs, state your approach, write simple code, test an edge case, give the complexity
SQL task	15–20	Joins, aggregation, window functions, NULL handling	Build in CTEs step by step, name the grain, check ties and NULLs
Pipeline architecture	40–50	Senior judgment: trade-offs, reliability, scale, cost	Requirements first, then diagram, then deep dives, with a reason for every choice
Your questions	5	Curiosity about the team	2–3 specific questions (section 7)

Scoring mindset

- They can only score what you say out loud. Think aloud through every step.
- A working simple solution beats an unfinished clever one. Get it working first, then improve it.
- When you don't know something, say what you would check and how. Never go silent and never bluff.

- In design, every choice needs a "because". Wrong-but-reasoned scores higher than right-but-unexplained.

4-day plan (Sat 7 – Tue 10 Nov)

About 3 hours a day, with half of that on pipeline design because that section is worth half the score. Every day ends with a timed practice round. I can run that as the interviewer.

Day	Pipeline design (about 90 min)	SQL (about 45 min)	Python (about 45 min)	Practice round with Claude
Sat 7	Learn the 6-step framework (section 5). Redraw the inDrive example from memory.	Patterns 1–3: dedup, top-N per group, running total	Patterns 1–3: count, group, dedup	1 SQL problem, 15 min timer
Sun 8	Deep dives: exactly-once, late data, CDC, backfill	Patterns 4–6: conditional aggregation, anti-join, gaps and islands	Patterns 4–6: parse, merge intervals, sessions	Mock design round A (CDC to warehouse), 40 min
Mon 9	Deep dives: data quality, monitoring, BigQuery cost, PII	Patterns 7–8 + the traps list	Patterns 7–9: sliding window, batches, flatten	Mock design round B (streaming dashboard), 40 min
Tue 10	Read through the follow-up questions and say the answers out loud	2 new problems, 15 min each	2 new problems, 15 min each	Full 90-min mock interview, then fix weak spots
Wed 11	30 min, morning only: read sections 5 and 6, then stop	–	–	Join at 12:20

Rules for practice

1. Always speak in English while you solve. It trains the language and the thinking-aloud habit together.
2. Write code in a plain editor with no autocomplete, because a live coding pad usually has none.
3. Use a timer. When it rings, stop and say what you would do next, exactly as you would in the real interview.

Python task (15–20 min)

A data-engineering Python task almost always means processing a list of records (dicts, JSON lines, log rows) with plain Python, no pandas. Learn these 9 patterns until you can type them without thinking. inDrive's job post also asks for OOP and clean architecture, so wrapping your solution in a small function or class with clear names earns extra points.

How to run the 15 minutes

1. **Clarify (2 min):** input format, output format, size of data, edge cases (empty input, duplicates, missing keys, ties).
2. **Say the approach (1 min):** "I'll loop once and keep a dict keyed by driver_id, so it's O(n) time and O(k) memory."
3. **Code (8 min):** simple names, small functions, no premature optimisation.
4. **Test (3 min):** run one normal example and one edge case out loud.
5. **Scale (1 min):** "For 100 GB I'd stream the file with a generator, or move it to PySpark with the same logic as a groupBy."

The 9 patterns

```
from collections import Counter, defaultdict, deque
from datetime import timedelta
import json

# 1. Count + top-k: top 3 cities by number of rides
def top_cities(rides, k=3):
    return Counter(r["city"] for r in rides).most_common(k)

# 2. Group + aggregate: average fare per driver
def avg_fare_per_driver(rides):
    totals = defaultdict(lambda: [0.0, 0])          # driver -> [sum, count]
    for r in rides:
        t = totals[r["driver_id"]]
```

```
        t[0] += r["fare"]
        t[1] += 1
    return {d: s / c for d, (s, c) in totals.items()}

# 3. Dedup, keep the latest version of each record
def latest_per_id(events):
    latest = {}
    for e in events:
        cur = latest.get(e["id"])
        if cur is None or e["updated_at"] > cur["updated_at"]:
            latest[e["id"]] = e
    return list(latest.values())

# 4. Parse JSON lines safely, count bad records instead of crashing
def parse_lines(lines):
    good, bad = [], 0
    for line in lines:
        try:
            rec = json.loads(line)
            if "ride_id" not in rec:
                raise ValueError("missing ride_id")
            good.append(rec)
        except (json.JSONDecodeError, ValueError):
            bad += 1                                  # in production: send to
a dead-letter queue
    return good, bad

# 5. Merge overlapping intervals: total online time of a driver
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged

# 6. Sessionize: a new session starts after a 30-minute gap
def count_sessions(timestamps, gap=timedelta(minutes=30)):
    ts = sorted(timestamps)
    if not ts:
        return 0
    return 1 + sum(1 for prev, cur in zip(ts, ts[1:]) if cur - prev > gap)
```

```
# 7. Sliding window: max rides in any 60-minute window
def max_in_window(timestamps, window=timedelta(hours=1)):
    q, best = deque(), 0
    for t in sorted(timestamps):
        q.append(t)
        while t - q[0] >= window:
            q.popleft()
        best = max(best, len(q))
    return best

# 8. Process a huge file in batches with a generator (constant memory)
def read_in_batches(path, size=10_000):
    batch = []
    with open(path) as f:
        for line in f:
            batch.append(line)
            if len(batch) == size:
                yield batch
                batch = []
    if batch:
        yield batch

# 9. Flatten nested JSON: {"a": {"b": 1}} -> {"a.b": 1}
def flatten(d, parent="", sep="."):
    out = {}
    for k, v in d.items():
        key = f"{parent}{sep}{k}" if parent else k
        if isinstance(v, dict):
            out.update(flatten(v, key, sep))
        else:
            out[key] = v
    return out
```

Quick facts to say when asked

- dict and set lookups are O(1) on average; sorting is O(n log n).
- Use a generator when the data doesn't fit in memory.
- A list is mutable, and a tuple is immutable, so only a tuple can be a dict key.
- To test, write `assert` lines for a normal case, an empty input and a duplicate/tie case.

SQL task (15–20 min)

Expect 1–2 questions on ride-hailing data, almost certainly needing a window function. The 8 patterns below cover most of what gets asked. inDrive's warehouse is BigQuery, so BigQuery syntax differences are noted in the comments.

Practice schema

```
rides(ride_id, passenger_id, driver_id, city_id, status,   -- 'completed' |
'cancelled' | ...
      proposed_fare, fare, requested_at, completed_at, updated_at)
bids(bid_id, ride_id, driver_id, bid_price, created_at, accepted)   --
inDrive fare negotiation
drivers(driver_id, city_id, signup_date)
```

How to run the 15 minutes

1. Repeat the question back and confirm the **grain** of the output: "one row per city per day, right?"
2. Ask about edge cases: ties, NULLs, cancelled rides, time zone.
3. Build it in CTEs, one step per CTE, and say what each one returns.
4. Check it against a 3-row example in your head before you say "done".

The 8 patterns

```
-- 1. Dedup: keep the latest version of each ride
SELECT * FROM (
  SELECT r.*, ROW_NUMBER() OVER (PARTITION BY ride_id ORDER BY updated_at
DESC) AS rn
  FROM rides_raw r
) t
WHERE rn = 1;
-- BigQuery: SELECT * FROM rides_raw QUALIFY ROW_NUMBER() OVER (PARTITION BY
ride_id ORDER BY updated_at DESC) = 1

-- 2. Top-N per group: top 3 drivers by revenue in each city
WITH driver_rev AS (
  SELECT city_id, driver_id, SUM(fare) AS revenue
  FROM rides
  WHERE status = 'completed'
  GROUP BY city_id, driver_id
)
SELECT city_id, driver_id, revenue
```

```
FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY city_id ORDER BY revenue DESC) AS
rnk
  FROM driver_rev
) t
WHERE rnk <= 3;      -- say why: DENSE_RANK keeps ties, ROW_NUMBER would drop
them

-- 3. Running total, 7-day moving average, change vs previous day
WITH daily AS (
  SELECT DATE(requested_at) AS d, COUNT(*) AS rides
  FROM rides
  GROUP BY 1
)
SELECT d, rides,
       SUM(rides) OVER (ORDER BY d)                                    AS
running_total,
       AVG(rides) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
AS avg_7d,
       rides - LAG(rides) OVER (ORDER BY d)                            AS
change_vs_prev_day
FROM daily;

-- 4. Conditional aggregation: completion rate per city
SELECT city_id,
       COUNT(*) AS requests,
       ROUND(100.0 * SUM(CASE WHEN status = 'completed' THEN 1 ELSE 0 END) /
COUNT(*), 1) AS completion_pct
FROM rides
GROUP BY city_id
HAVING COUNT(*) >= 100           -- ignore tiny cities
ORDER BY completion_pct DESC;

-- 5. Anti-join: drivers with no ride in the last 30 days
SELECT d.driver_id
FROM drivers d
LEFT JOIN rides r
  ON  r.driver_id = d.driver_id
  AND r.requested_at >= CURRENT_DATE - INTERVAL '30' DAY   -- filter in ON,
not WHERE!
WHERE r.ride_id IS NULL;
-- BigQuery: TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY)
```

```
-- 6. Gaps and islands: longest streak of consecutive active days per driver
WITH active_days AS (
  SELECT DISTINCT driver_id, DATE(completed_at) AS d
  FROM rides WHERE status = 'completed'
), numbered AS (
  SELECT driver_id, d,
         ROW_NUMBER() OVER (PARTITION BY driver_id ORDER BY d) AS rn
  FROM active_days
)
SELECT driver_id, MIN(d) AS streak_start, MAX(d) AS streak_end, COUNT(*) AS
days
FROM numbered
GROUP BY driver_id, d - rn * INTERVAL '1' DAY      -- same value = same
streak
ORDER BY days DESC;
-- BigQuery: GROUP BY driver_id, DATE_SUB(d, INTERVAL rn DAY)

-- 7. Retention: of passengers whose first ride was on day X, how many rode
again within 7 days?
WITH first_ride AS (
  SELECT passenger_id, MIN(DATE(requested_at)) AS first_day
  FROM rides WHERE status = 'completed'
  GROUP BY passenger_id
)
SELECT f.first_day,
       COUNT(DISTINCT f.passenger_id) AS new_passengers,
       COUNT(DISTINCT CASE WHEN DATE(r.requested_at) >  f.first_day
                            AND DATE(r.requested_at) <= f.first_day + 7
                           THEN f.passenger_id END) AS retained_7d
FROM first_ride f
LEFT JOIN rides r ON r.passenger_id = f.passenger_id AND r.status =
'completed'
GROUP BY f.first_day;
-- BigQuery: DATE_ADD(f.first_day, INTERVAL 7 DAY)

-- 8. inDrive-specific: how often does the accepted bid beat the passenger's
proposed fare?
SELECT r.city_id,
       AVG(CASE WHEN b.bid_price > r.proposed_fare THEN 1.0 ELSE 0 END) AS
share_above_proposed,
       AVG(b.bid_price / NULLIF(r.proposed_fare, 0) - 1)              AS
```

```
avg_markup
FROM rides r
JOIN bids b ON b.ride_id = r.ride_id AND b.accepted
GROUP BY r.city_id;
```

Traps they check for

- `COUNT(*)` counts rows; `COUNT(col)` skips NULLs; `COUNT(DISTINCT col)` after a join avoids double-counting.
- A filter on the right table of a LEFT JOIN belongs in `ON`. In `WHERE` it silently turns the join into an INNER JOIN.
- Integer division: write `100.0 *` or `CAST(... AS FLOAT64)`.
- Divide by zero: wrap the divisor in `NULLIF(x, 0)`.
- Ties: say which you chose, `ROW_NUMBER`, `RANK` or `DENSE_RANK`, and why.
- Order of execution: FROM → WHERE → GROUP BY → HAVING → window functions → SELECT → ORDER BY. That's why you can't filter a window function in `WHERE`.

Pipeline architecture (40–50 min)

This section decides the interview. Drive it with the same 6 steps every time, so the interviewer sees structure and you never get lost. Their stack is Kafka, Pub/Sub, Debezium, Spark, Beam, Airflow 2, BigQuery, GCS, S3, Kubernetes and Terraform. Design with those names, and connect each one to the AWS service you have used.

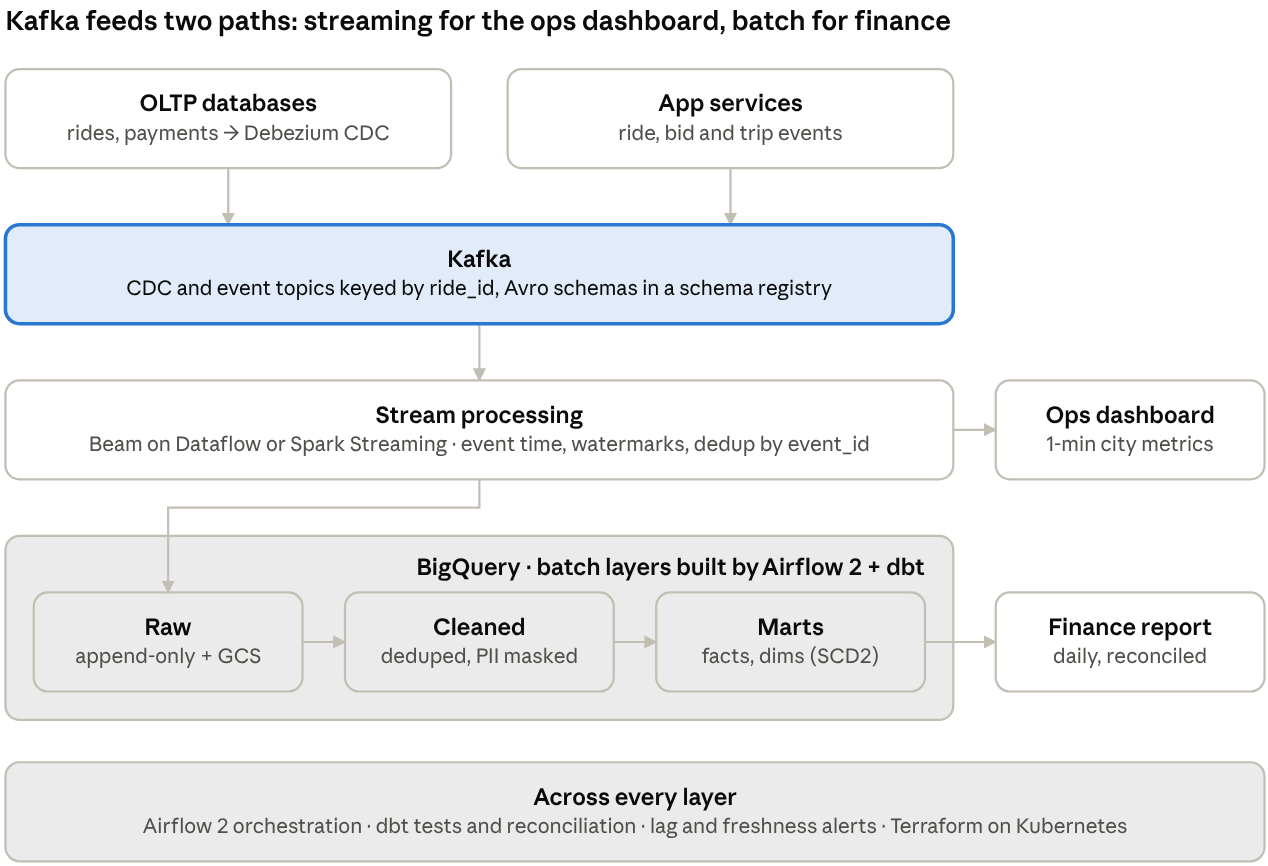
The 6-step framework

1. **Clarify requirements (5–7 min).** Don't draw anything yet. Ask:
   - Who uses the data and for what (dashboard, finance report, ML, fraud)?
   - Sources: databases (CDC), app events, third-party APIs, files?
   - Volume: events per second at peak, GB per day, growth?
   - Latency: seconds, minutes, hourly or daily?
   - Correctness: is it money? Then duplicates and losses are not acceptable.
   - History and retention, PII, compliance, cost limits.
   - Then **state your assumptions with numbers**: "I'll assume 5k events per second at peak, a 1-minute dashboard and a daily finance report."
2. **High-level diagram (5 min).** Sources → ingestion → raw layer → cleaned layer → marts → consumers. Orchestration, data quality and monitoring run across all of them.

3. **Walk through each layer (15–20 min).** For each layer, name the tool, why you chose it, and the alternative you rejected.
4. **Data model (5–10 min).** Facts and dimensions, the grain, SCD Type 2 where history matters, partitioning and clustering.
5. **Reliability (5–10 min).** Idempotency, late data, schema changes, backfills, data quality, alerting, cost.
6. **Wrap up (2 min).** The main trade-off you made and what you'd add in version 2.

Worked example

The case: "Design a pipeline for ride and bid data that powers a near-real-time city operations dashboard and a daily finance report."



Worked example architecture · streaming and batch paths

Both paths start from the same Kafka topics. The streaming path keeps the dashboard within a minute. The batch path rebuilds trusted, reconciled tables for finance.

Layer	Choice	Why	What you know on AWS
Database changes	Debezium CDC from the rides/payments OLTP databases into Kafka	Captures every insert, update and delete without loading the source database	DMS, or Kinesis + Lambda
App events	Services publish ride_requested, bid_created and trip_completed events to Kafka, with Avro/Protobuf and a schema registry	Ordered per key (ride_id); replayable; the schema is a contract	Kinesis Data Streams / MSK
Streaming path	Beam on Dataflow, or Spark Structured Streaming, using event time, windows and watermarks	Per-minute aggregates per city for the dashboard; handles late events	Kinesis Data Analytics / Glue streaming
Raw layer	Append-only raw tables in BigQuery, plus an archive in GCS (Parquet, partitioned by date)	Immutable history, so anything can be rebuilt	S3 raw zone
Cleaned layer	Dedup on event_id, type casts, PII masking, MERGE for CDC upserts	One trusted version of each ride	Glue jobs into curated S3
Marts	fct_ride_request, fct_bid, fct_trip, dim_driver (SCD2), dim_passenger, dim_city, dim_date, built with dbt/Dataform	Simple, fast queries for analysts and finance	Athena views / Redshift

Layer	Choice	Why	What you know on AWS
Orchestration	Airflow 2 on Kubernetes: sensors, retries, SLAs, backfills by logical date	Batch path and dependency management	Step Functions / Glue Workflows / MWAA
Quality and monitoring	dbt tests, freshness checks, row-count and amount reconciliation against the source, Kafka consumer lag, alerts	Finance numbers must match the source	Glue Data Quality, CloudWatch
Infrastructure	Terraform, Kubernetes, CI/CD with GitHub Actions	Repeatable environments, code review on pipelines	CloudFormation / CDK

Data model to say out loud: fct_bid has one row per driver offer on a request. fct_trip has one row per completed trip. Partition both by event date. Cluster by city_id and driver_id. dim_driver is SCD Type 2 because rating, vehicle and status change over time, and a finance report must use the values that were true on the trip date.

Deep-dive answers to have ready

Topic	Your answer in one breath
Exactly-once	"Kafka gives at-least-once in practice, so I make writes idempotent: every event has an event_id, and I dedupe or MERGE on it. Duplicates can arrive but never land twice."
Late-arriving data	"Streaming: event-time windows with a watermark and allowed lateness. Batch: each run reprocesses the last 3 days of partitions, so late rows are picked up."
Backfill	"Every task is parameterised by Airflow's logical date and overwrites its own partition. A backfill is just rerunning those dates, safely."

Topic	Your answer in one breath
CDC deletes and updates	"Debezium emits op = c/u/d. I MERGE on the primary key, keep deletes as a soft-delete flag, and keep the full change log in raw."
Schema change	"A schema registry with backward-compatible rules: adding a field is fine, renaming breaks the producer's CI. Unknown fields go to raw, so nothing is lost."
Bad records	"Validate at ingestion. Failures go to a dead-letter topic or table with the error, plus an alert. Never block the whole pipeline for one bad row."
Data quality	"unique and not_null on keys, accepted_values on status, relationships between facts and dims, freshness SLAs, and daily reconciliation of counts and amounts against the source."
Monitoring	"Consumer lag, job duration and failures, data freshness per table, row-count anomalies, and BigQuery cost per team. Alerts go to on-call."
BigQuery cost and speed	"Partition by date with require_partition_filter, cluster by the columns we filter on, never SELECT *, use incremental models and pre-aggregated tables for dashboards."
PII and fintech	"Tokenise phone numbers and documents in the cleaned layer, apply column-level policy tags, and keep a deletion process for privacy requests. I did this kind of governance in banking."
Streaming or batch?	"Stream only what needs seconds or minutes (ops dashboard, fraud). Everything else is batch, because it's cheaper and simpler to fix."
10× traffic spike	"Kafka absorbs it as a buffer. Add partitions and consumers, autoscale Dataflow, and alert on lag instead of dropping data."

Other cases they might give you

- **CDC from MySQL/Postgres to BigQuery.** The twist: ordering, deletes, MERGE cost. Answer with Debezium → Kafka → raw change log → periodic MERGE into current-state tables.
- **Clickstream / app analytics.** The twist: huge volume and sessionization. Answer with Pub/Sub or Kafka → Beam → partitioned raw tables → a sessions model.
- **Real-time fraud signals.** The twist: seconds of latency and state. Answer with stream processing and keyed state per user/device, with features served to a model.
- **Payments reconciliation / driver payouts.** The twist: correctness over speed. Answer with batch, double-entry checks, reconciliation reports and audit trails. This is your strongest story from banking.
- **Third-party API ingestion.** The twist: rate limits, pagination, retries. Answer with an Airflow task that fetches incrementally using a watermark column, lands raw JSON and retries with backoff.

Follow-up questions to practise out loud

1. A job failed halfway through. What happens when it reruns?
2. Finance says yesterday's revenue doesn't match the payments system. How do you investigate?
3. How would you reprocess 6 months of data after a logic bug?
4. Why Kafka and not Pub/Sub here, or the other way round?
5. How do you test a pipeline before it reaches production?
6. What would you change if latency had to drop from 1 hour to 1 minute?

Introduction and English phrases

Practise the intro until it takes 60 seconds and sounds natural. Practise it from the key points, not as a script you memorise word for word.

Your 60-second introduction (edit to sound like you)

> Hi, I'm Ahmed. I'm a senior data engineer with more than 20 years in IT, and for the last several years I've focused on data platforms for digital banking at Boubyan Digital Factory and Nomo.
>
> Day to day, I build serverless pipelines on AWS: streaming data in with Kinesis and Lambda, transforming it with Glue and PySpark, storing it in S3 and serving it through Athena. Banking taught me to care a lot about correctness: reconciliation, data quality checks and audit trails. Last month I passed the AWS Certified Data Engineer – Associate exam.
>
> What attracts me to inDrive is the scale of real-time marketplace data, and the fintech side of the business, where my banking experience fits directly. I'm also keen to work more deeply with Kafka, Spark and Airflow on your platform.

Phrases for thinking out loud

Moment	Say this
Before you start	"Before I start, can I ask a few questions about the input?"
Stating an assumption	"I'll assume the timestamps are in UTC. Please tell me if that's wrong."
Explaining your plan	"My approach is to… first…, then…. That gives O(n) time."
Buying time	"Let me think about that for a moment."
Stuck	"I'm not sure about the exact syntax. In BigQuery I believe it's…, and I'd confirm it in the docs."
Don't know a tool	"I haven't used Beam in production, but I've built the same pattern with Glue streaming. The concepts map like this…"
Choosing between options	"There are two options here. A is simpler, B scales better. Given the 1-minute latency, I'd pick B because…"
Checking in	"Does that answer your question, or should I go deeper on this part?"

Moment	Say this
Finishing	"To summarise: the main trade-off was…, and in version 2 I'd add…"

English corrections to remember

- Say "domain teams **own** their data", not "interview their data".
- Say "Data Mesh **is an architecture** / an operating model", not "is a style".
- Say "I **have** 20 years of experience", not "I am 20 years experience".
- Say "**depends on**", not "depends from".
- Say "I **was responsible for**", not "I was responsible on".

Questions to ask them and day-of checklist

Pick 2–3 questions. Listen in the introduction for what they already answered, and skip those.

Questions to ask

1. Which data domain would this role work on first, and what's the biggest pain point there today?
2. How do you split workloads between GCP and AWS, and where is that heading?
3. Do you model with dbt or Dataform, and how are pipelines tested before production?
4. Who owns data quality: the producing teams or the data platform team?
5. What would a great first 90 days look like for this person?

Before Wednesday

- [ ] Ask the recruiter which coding tool they use (CoderPad, a shared doc, your own IDE?), whether you can run the code, and which SQL dialect they expect
- [ ] Ask which drawing tool to use for the design part, or have Excalidraw open and tested
- [ ] Test camera, microphone, screen sharing and internet; have a mobile hotspot as backup
- [ ] Quiet room, water, paper and pen for quick notes

On the day

- [ ] 30-minute light review in the morning, then stop studying
- [ ] Close notifications and other apps

- [ ] Join at 12:20, 10 minutes early
- [ ] Smile, speak slowly, think out loud
- [ ] After the interview, write down every question they asked while you still remember them

Sources

- inDrive careers – Data Engineer (Data Delivery): tech stack, responsibilities and requirements used throughout this doc.
- All Python snippets and SQL patterns were run on sample data (Python 3 and DuckDB) before being added here. Only the BigQuery-specific lines in the comments are untested.